

Comment Claude Code isole ses opérations au niveau OS

Principe : l'OS comme périmètre de sécurité

Depuis v2.1.0, Claude Code intègre un sandbox natif qui isole l'exécution des commandes bash au niveau du noyau. L'idée centrale est d'inverser la logique habituelle : plutôt que de valider chaque commande par une confirmation utilisateur, on laisse Claude opérer librement à l'intérieur de frontières OS-enforced. Le sandbox devient le gardien, pas le système de permissions.

Activation : `/sandbox` dans Claude Code (menu interactif).

Primitives OS selon la plateforme

macOS : Seatbelt. Intégré au système, aucune installation requise. Utilise le framework TrustedBSD Mandatory Access Control pour filtrer les appels système au niveau kernel. Mesurer la surcharge avec vos commandes.

Linux / WSL2 : bubblewrap. Nécessite l'installation de deux paquets.

```
sudo apt-get install bubblewrap socat # Ubuntu/Debian
sudo dnf install bubblewrap socat # Fedora
```

Crée un namespace isolé (mount, network, PID, IPC) pour chaque commande exécutée. Les performances dépendent du travail et de la plateforme.

WSL1 : non supporté. **Windows natif** : non pris en charge ; utiliser WSL2.

Isolation filesystem

La politique par défaut est asymétrique : lecture sur le système, écriture dans le répertoire de travail, le répertoire temporaire de l'utilisateur et les répertoires explicitement ajoutés, sous réserve des chemins protégés et des règles configurées.

Accès	Périmètre
Lecture	Tout l'ordinateur (sauf deny rules)
Écriture	Répertoire de travail, temporaire et répertoires ajoutés, sous réserve des restrictions
Exécution	Commandes dans l'environnement sandboxé

Pour restreindre la lecture de répertoires sensibles (SSH, credentials cloud), utiliser les `permissions.deny` dans

`settings.json` :

```
{
  "permissions": {
    "deny": [
      "Read(~/.ssh/**)",
      "Read(~/.aws/**)",
      "Edit(~/.ssh/**)",
      "Edit(~/.aws/**)"
    ]
  }
}
```

Isolation réseau

Toutes les connexions réseau des commandes sandboxées passent par un proxy SOCKS5 qui tourne hors sandbox. Le proxy applique un filtrage par domaine sans inspecter le contenu du trafic (pas de deep packet inspection).

Deux modes selon `sandbox.network.strictAllowlist` : permissif (défaut, un domaine hors liste déclenche une confirmation) ou strict (`strictAllowlist: true` , échec immédiat sans confirmation).

```
{
  "sandbox": {
    "network": {
      "strictAllowlist": true,
      "allowedDomains": [
        "api.anthropic.com",
        "*.npmjs.org",
        "github.com"
      ]
    }
  }
}
```

Deux modes de fonctionnement

Auto-allow : les commandes bash dans le sandbox sont automatiquement approuvées. Les commandes incompatibles avec le sandbox (ex : `docker`) déclenchent un fallback vers le flux de permissions normal. Recommandé pour le développement quotidien.

Regular permissions : les commandes sandboxées suivent les règles normales de permission ; les autorisations existantes restent applicables. Utile pour les environnements sensibles ou les bases de code non fiables.

L'escape hatch : `dangerouslyDisableSandbox`

Certains outils sont incompatibles avec le sandboxing car ils nécessitent un accès à des ressources système spécifiques (`docker` via `/var/run/docker.sock` , `watchman` via les filesystem watch APIs). Claude peut réessayer avec ce paramètre. La reprise suit le flux normal de permissions, selon le mode actif ; une règle ask peut imposer une confirmation. Pour désactiver complètement l'escape hatch dans les environnements critiques :

```
{
  "sandbox": {
    "allowUnsandboxedCommands": false,
    "failIfUnavailable": true
  }
}
```

`failIfUnavailable: true` (v2.1.83+) : fait échouer la session si le sandbox ne peut pas démarrer, plutôt que de continuer sans isolation. Recommandé pour les environnements de production.